



LABORATORIUM PEMBELAJARAN ILMU KOMPUTER
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

PRAKTIKUM EMBEDDED SYSTEM

SEMESTER : GENAP

TAHUN : 2025/2026

BAB IV

JUDUL BAB : RTOS
NAMA : MUHAMMAD NAUFAL MATHARA
RAHMAN
NIM : 245150307111006
TANGGAL PELAKSANAAN : 22 APRIL 2026
ASISTEN : - MUHAMMAD WAHYU CAHYO P.
- DAVIN KENAZ WIDIANANDA TAPPO
KOORDINATOR ASISTEN : DINDA INAYATUR ROSYIDAH
DOSEN PENGAMPU : EKO SETIAWAN, S.T., M.T., M.Eng., Ph.D.

Pengesahan

Tanggal:

Asisten



LABORATORIUM PEMBELAJARAN ILMU KOMPUTER
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

BAB : RTOS
NAMA : MUHAMMAD NAUFAL MATHARA RAHMAN
NIM : 245150307111006
TANGGAL : 22/04/2026
ASISTEN : - MUHAMMAD WAHYU CAHYO PRASETYO
- DAVIN KENAZ WIDIANANDA TAPPO

LAPORAN

Percobaan 4.1: Penjadwalan Multitasking Tanpa RTOS

A. Source Code

Percobaan41.ino

```
1 // define function
2 void TaskBlink();
3 void TaskAnalogRead();
4 void TaskDigitalRead();
5
6 void setup() {
7     // initialize serial communication at 9600 bits per second:
8     Serial.begin(9600);
9     while (!Serial) {
10         ; // wait for serial port to connect.
11     }
12     pinMode(8, OUTPUT);
13     pinMode(2, INPUT);
14     pinMode(13, OUTPUT);
15 }
16
17 void loop() {
18     TaskBlink();
19     TaskAnalogRead();
20     TaskDigitalRead();
21 }
22
23 /*-----*/
24 /*----- Function -----*/
25 /*-----*/
26
27 void TaskBlink() {
28     /*
29     Blink Turns on an LED on for one second, then off for one
30     second, repeatedly.
31     */
32     digitalWrite(8, HIGH); // turn the LED on
33     delay(1000);           // wait for one second
34     digitalWrite(8, LOW);  // turn the LED off
```

```

35     delay(1000);           // wait for one second
36 }
37
38 void TaskAnalogRead() {
39     /*
40     AnalogReadSerial
41     Reads an analog input on pin A0, prints the result to the
42     serial
43     monitor. Graphical representation is available using serial
44     plotter
45     (Tools > Serial Plotter menu)
46     */
47     // read the input on analog pin 0:
48     int sensorValue = analogRead(A0);
49     // print out the value you read:
50     Serial.println(sensorValue);
51     delay(15); // delay (15ms) in between reads for stability
52 }
53
54 void TaskDigitalRead() {
55     /*
56     DigitalRead
57     Turns on and off a light emitting diode(LED) connected to
58     digital pin 13 (onboard/built in), when pressing a
59     pushbutton
60     attached to pin 2.
61     */
62     // check if the pushbutton is pressed.
63     if (digitalRead(2) == HIGH) {
64         // turn LED on:
65         digitalWrite(13, HIGH);
66     } else {
67         // turn LED off:
68         digitalWrite(13, LOW);
69     }
70 }
71

```

B. Penjelasan Kode

Percobaan41.ino

void TaskBlink(); Mendeklarasikan fungsi untuk membuat LED berkedip.

void TaskAnalogRead(); Mendeklarasikan fungsi untuk membaca nilai sensor analog.

void TaskDigitalRead(); Mendeklarasikan fungsi untuk membaca status tombol digital.

void setup() Fungsi inisialisasi yang hanya dijalankan satu kali saat Arduino pertama kali dinyalakan.

Serial.begin(9600); Memulai komunikasi serial dengan komputer pada kecepatan 9600 baud rate.

```
while (!Serial) Menunggu hingga koneksi port serial berhasil tersambung.  
pinMode(8, OUTPUT); Mengatur pin digital 8 agar berfungsi sebagai output (misalnya  
untuk LED).  
pinMode(2, INPUT); Mengatur pin digital 2 agar berfungsi sebagai input (misalnya  
untuk tombol).  
pinMode(13, OUTPUT); Mengatur pin digital 13 (LED bawaan board) agar berfungsi  
sebagai output.  
void loop() Fungsi utama yang akan terus dijalankan secara berulang-ulang/tanpa batas.  
TaskBlink(); Memanggil dan mengeksekusi perintah di dalam fungsi TaskBlink.  
TaskAnalogRead(); Memanggil dan mengeksekusi perintah di dalam fungsi  
TaskAnalogRead.  
TaskDigitalRead(); Memanggil dan mengeksekusi perintah di dalam fungsi  
TaskDigitalRead.  
void TaskBlink() Awal dari definisi isi fungsi TaskBlink.  
digitalWrite(8, HIGH); Memberikan tegangan pada pin 8 (menyalakan LED).  
delay(1000); Menghentikan sementara program selama 1000 milidetik (1 detik).  
digitalWrite(8, LOW); Memutus tegangan pada pin 8 (mematikan LED).  
delay(1000); Menghentikan sementara program selama 1000 milidetik (1 detik).  
void TaskAnalogRead() Awal dari definisi isi fungsi TaskAnalogRead.  
int sensorValue = analogRead(A0); Membaca nilai dari pin analog A0 dan  
menyimpannya di dalam variabel bilangan bulat bernama sensorValue.  
Serial.println(sensorValue); Mengirim dan menampilkan nilai dari variabel sensorValue  
ke Serial Monitor, lalu pindah baris (Enter).  
delay(15); Memberikan jeda 15 milidetik untuk menstabilkan konverter  
analog-ke-digital sebelum pembacaan berikutnya.  
void TaskDigitalRead() Awal dari definisi isi fungsi TaskDigitalRead.  
if (digitalRead(2) == HIGH) Mengecek apakah status pin 2 sedang menerima  
tegangan/sinyal HIGH (tombol sedang ditekan).  
digitalWrite(13, HIGH); Menyalakan LED pada pin 13 jika kondisi tombol pada pin 2  
di atas terpenuhi (ditekan).  
else Kondisi sebaliknya, jika tombol pada pin 2 tidak ditekan (menerima sinyal LOW).
```

`digitalWrite(13, LOW);` Mematikan LED pada pin 13 karena kondisi tombol tidak ditekan.

C. Analisis 4.1

1. *Lakukan perubahan nilai input pada Arduino dengan memutar potensiometer (dan mengamati hasil pembacaannya pada serial monitor), sekaligus melakukan penekanan berulang-ulang pada push button, juga sekaligus amati output pada blinking LED*

Jawab:

Saat potensiometer diputar dan tombol ditekan bersamaan, respon sistem terasa sangat lambat. Pembacaan nilai ADC di serial monitor tertahan dan lambat update-nya. Begitu juga dengan push button, sistem sering kali tidak mendeteksi penekanan tombol kecuali tombol ditahan agak lama sampai giliran pembacaannya tiba.

2. *Jelaskan bagaimana respon masing-masing task ketika dijalankan, mengenai apakah setiap task selalu merespon tepat waktu (kedipan LED, hasil baca ADC, hasil LED sesuai penekanan push button)*

Jawab:

Task tidak merespon tepat waktu. Kedipan LED memang berjalan normal, tapi task baca ADC dan push button merespon sangat telat. Hal ini terjadi karena sistem terjebak pada fungsi `delay()` di task blink LED. Selama proses delay tersebut berlangsung, mikrokontroler diam dan tidak bisa mengeksekusi task lain sama sekali.

Percobaan 4.2: Penjadwalan Multitasking dengan RTOS

A. Source Code

Percobaan42.ino

```
1  #include <Arduino_FreeRTOS.h>
2
3  // define tasks
4  void TaskBlink( void *pvParameters );
5  void TaskAnalogRead( void *pvParameters );
6  void TaskDigitalRead( void *pvParameters );
7
8  void setup() {
9
10     // initialize serial communication at 9600 bits per second:
11     Serial.begin(9600);
12
13     while (!Serial) {
```

```

14         ; // wait for serial port to connect.
15     }
16
17     // Now set up tasks to run independently.
18     xTaskCreate(
19         TaskBlink
20         , "Blink" // A name just for humans
21         , 128 // stack size
22         , NULL
23         , 2 // Priority, with 3 being the highest, 0 being the
24 lowest.
25         , NULL );
26
27     xTaskCreate(
28         TaskAnalogRead
29         , "AnalogRead"
30         , 128
31         , NULL
32         , 1 // Priority
33         , NULL );
34
35     xTaskCreate(
36         TaskDigitalRead
37         , "DigitalRead"
38         , 128 // Stack size
39         , NULL
40         , 1 // Priority
41         , NULL );
42
43     // Now the task scheduler is automatically started.
44 }
45
46 void loop()
47 {
48     // Empty. Things are done in Tasks.
49 }
50
51 /*-----*/
52 /*----- Tasks -----*/
53 /*-----*/
54
55 void TaskBlink(void *pvParameters) // This is a task.
56 {
57     (void) pvParameters;
58
59     /*
60     Blink Turns on an LED on for one second, then off for one
61     second, repeatedly.
62     */
63
64     // initialize digital pin 8 as an output.
65     pinMode(8, OUTPUT);
66

```

```

67     for (;;) // A Task shall never return or exit.
68     {
69         digitalWrite(8, HIGH); // turn the LED on
70         vTaskDelay( 1000 / portTICK_PERIOD_MS ); // one second
71         digitalWrite(8, LOW); // turn the LED off
72         vTaskDelay( 1000 / portTICK_PERIOD_MS ); // one second
73     }
74 }
75
76 void TaskAnalogRead(void *pvParameters) // This is a task.
77 {
78     (void) pvParameters;
79
80     /*
81     AnalogReadSerial
82     Reads an analog input on pin A0, prints the result to the
83     serial
84     monitor.
85     Graphical representation is available using serial plotter
86     (Tools > Serial Plotter menu)
87     */
88     for (;;)
89     {
90         // read the input on analog pin 0:
91         int sensorValue = analogRead(A0);
92         // print out the value you read:
93         Serial.println(sensorValue);
94         // one tick delay (15ms) in between reads for stability
95         vTaskDelay(1);
96     }
97 }
98
99 void TaskDigitalRead(void *pvParameters) // This is a task.
100 {
101     (void) pvParameters;
102     /*
103     DigitalRead
104     Turns on and off a light emitting diode(LED) connected to
105     digital pin 13 (onboard/built in), when pressing a
106     pushbutton
107     attached to pin 2.*/
108     pinMode(2, INPUT);
109     pinMode(13, OUTPUT);
110
111     for (;;)
112     {
113         if (digitalRead(2) == HIGH) {
114             // turn LED on:
115             digitalWrite(13, HIGH);
116         } else {
117             // turn LED off:
118             digitalWrite(13, LOW);
119         }
120     }
121 }

```

120	}
121	}
122	

B. Penjelasan Kode

Percobaan42.ino

#include <Arduino_FreeRTOS.h> Memasukkan pustaka FreeRTOS agar Arduino dapat menjalankan sistem operasi waktu nyata (multitasking).

void TaskBlink(void *pvParameters); Mendeklarasikan tugas (task) untuk membuat LED berkedip.

void TaskAnalogRead(void *pvParameters); Mendeklarasikan tugas (task) untuk membaca sensor analog.

void TaskDigitalRead(void *pvParameters); Mendeklarasikan tugas (task) untuk membaca input digital (tombol).

void setup() Fungsi inisialisasi yang dijalankan sekali saat Arduino pertama kali menyala.

Serial.begin(9600); Memulai komunikasi serial dengan komputer pada kecepatan 9600 baud rate.

while (!Serial) Menunggu hingga koneksi port serial berhasil tersambung.

xTaskCreate(Memanggil fungsi dari FreeRTOS untuk membuat sebuah tugas (task) baru.

TaskBlink Nama fungsi yang akan dijalankan sebagai tugas pertama.

, "Blink" Memberikan label nama "Blink" untuk tugas ini (berguna bagi manusia untuk pembacaan/debugging).

, 128 Mengalokasikan ukuran memori (stack size) sebesar 128 word untuk tugas ini.

, NULL Tidak ada parameter tambahan yang dikirimkan ke dalam tugas.

, 2 Menetapkan prioritas tugas menjadi 2 (prioritas lebih tinggi dari task lainnya).

, NULL); Tidak menggunakan handle (penunjuk) untuk memantau tugas ini.

xTaskCreate(Membuat tugas (task) yang kedua.

TaskAnalogRead Nama fungsi yang dijalankan sebagai tugas kedua.

, "AnalogRead" Memberikan label nama "AnalogRead" pada tugas ini.

, 128 Mengalokasikan ukuran memori sebesar 128 word.

, NULL Tidak ada parameter tambahan.

, 1 Menetapkan prioritas tugas menjadi 1.
, NULL); Tidak menggunakan handle.
xTaskCreate(Membuat tugas (task) yang ketiga.
TaskDigitalRead Nama fungsi yang dijalankan sebagai tugas ketiga.
, "DigitalRead" Memberikan label nama "DigitalRead" pada tugas ini.
, 128 Mengalokasikan ukuran memori sebesar 128 word.
, NULL Tidak ada parameter tambahan.
, 1 Menetapkan prioritas tugas menjadi 1.
, NULL); Tidak menggunakan handle.
void loop() Fungsi utama bawaan Arduino. Dibiarkan kosong karena pengulangan program kini diambil alih sepenuhnya oleh tugas-tugas (tasks) FreeRTOS.
void TaskBlink(void *pvParameters) Definisi isi perintah dari tugas TaskBlink.
(void) pvParameters; Mencegah munculnya peringatan (warning) dari compiler karena parameter pvParameters tidak dipakai di dalam fungsi ini.
pinMode(8, OUTPUT); Mengatur pin 8 agar berfungsi sebagai output.
for (;;) Membuat perulangan tanpa batas (infinite loop) yang hukumnya wajib ada pada setiap task FreeRTOS agar tugas tidak pernah berhenti/keluar.
digitalWrite(8, HIGH); Memberikan tegangan pada pin 8 (menyalakan LED).
vTaskDelay(1000 / portTICK_PERIOD_MS); Menjeda tugas ini selama 1000 milidetik (1 detik) menggunakan sistem tick dari FreeRTOS, sehingga task lain bisa berjalan selama jeda ini.
digitalWrite(8, LOW); Memutus tegangan pada pin 8 (mematikan LED).
vTaskDelay(1000 / portTICK_PERIOD_MS); Menjeda kembali tugas ini selama 1 detik.
void TaskAnalogRead(void *pvParameters) Definisi isi perintah dari tugas TaskAnalogRead.
(void) pvParameters; Mencegah peringatan compiler untuk variabel parameter yang tidak digunakan.
for (;;) Membuat perulangan tanpa batas untuk tugas membaca analog.
int sensorValue = analogRead(A0); Membaca nilai tegangan dari pin analog A0 dan menyimpannya di variabel sensorValue.

Serial.println(sensorValue); Mengirim dan mencetak nilai sensorValue ke Serial Monitor.

vTaskDelay(1); Memberikan jeda waktu 1 tick RTOS (sekitar 15ms) untuk menstabilkan pembacaan analog dan memberi giliran pada task lain.

void TaskDigitalRead(void *pvParameters) Definisi isi perintah dari tugas TaskDigitalRead.

(void) pvParameters; Mencegah peringatan compiler untuk variabel parameter yang tidak digunakan.

pinMode(2, INPUT); Mengatur pin 2 agar berfungsi sebagai input (tombol).

pinMode(13, OUTPUT); Mengatur pin 13 (LED internal) agar berfungsi sebagai output.

for (;;) Membuat perulangan tanpa batas untuk tugas membaca tombol digital.

if (digitalRead(2) == HIGH) Mengecek apakah status pin 2 sedang menerima tegangan/sinyal HIGH (tombol ditekan).

digitalWrite(13, HIGH); Menyalakan LED pada pin 13 karena tombol ditekan.

else Kondisi alternatif jika tombol pada pin 2 tidak ditekan (LOW).

digitalWrite(13, LOW); Mematikan LED pada pin 13 karena tombol dilepas.

C. Analisis 4.2

1. *Lakukan perubahan nilai input pada Arduino dengan memutar potensiometer (dan mengamati hasil pembacaannya pada serial monitor), sekaligus melakukan penekanan berulang-ulang pada push button, juga sekaligus amati output pada blinking LED*

Jawab:

Berbeda jauh dengan percobaan sebelumnya, kali ini saat potensiometer diputar dan push button ditekan berulang-ulang, sistem merespon dengan sangat cepat. Data di serial monitor update secara realtime tanpa terlihat adanya jeda..

2. *Jelaskan bagaimana respon masing-masing task ketika dijalankan, mengenai apakah setiap task selalu merespon tepat waktu (kedipan LED, hasil baca ADC, hasil LED sesuai penekanan push button).*

Jawab:

Semua task kini merespon tepat waktu. LED tetap berkedip sesuai interval, hasil baca ADC langsung muncul saat diputar, dan status tombol terekam seketika saat ditekan. Ini karena menggunakan vTaskDelay dari FreeRTOS.

Percobaan 4.3: Penggunaan Semaphore

A. Source Code

Percobaan43.ino

```
1  #include <Arduino_FreeRTOS.h>
2  #include <semphr.h> // add the FreeRTOS functions for
3  Semaphores
4
5  /* Declare a mutex Semaphore Handle which we will use to
6  manage the
7  Serial Port. It will be used to ensure only one Task is
8  accessing
9  this resource at any time.*/
10 SemaphoreHandle_t xSerialSemaphore;
11
12 // define two Tasks for DigitalRead & AnalogRead
13 void TaskDigitalRead( void *pvParameters );
14 void TaskAnalogRead( void *pvParameters );
15
16 // the setup function runs once when you press reset or power
17 the board
18 void setup() {
19     // initialize serial communication at 9600 bits per second:
20     Serial.begin(9600);
21     while (!Serial) {
22         ; // wait for serial port to connect.
23     }
24
25     /* Semaphores are useful to stop a Task proceeding, where
26     it
27     should be paused to wait, because it is sharing a resource,
28     such as
29     the Serial port. Semaphores should only be used whilst the
30     scheduler
31     is running, but we can set it up here.*/
32     if ( xSerialSemaphore == NULL )
33         // confirm that the Serial Semaphore has not already been
34         created.
35         {
36             xSerialSemaphore = xSemaphoreCreateMutex();
37             // Create a mutex semaphore we will use to manage the
38             Serial Port
39             if ( ( xSerialSemaphore ) != NULL )
40                 xSemaphoreGive( ( xSerialSemaphore ) );
41             // Make the Serial Port available for use, by "Giving"
42             the Semaphore.
43         }
44
45     // Now set up two Tasks to run independently.
46     xTaskCreate(
```

```

47     TaskDigitalRead
48     , "DigitalRead" // A name just for humans
49     , 128 // Stack size
50     , NULL //Parameters for the task
51     , 2 // Priority, 3 being the highest, and 0 being the
52 lowest.
53     , NULL ); //Task Handle
54
55     xTaskCreate(
56         TaskAnalogRead
57         , "AnalogRead"
58         , 128
59         , NULL
60         , 1
61         , NULL );
62
63     // Now the Task scheduler is automatically started.
64 }
65
66 void loop()
67 {
68     // Empty. Things are done in Tasks.
69 }
70
71 /*-----*/
72 /*----- Tasks -----*/
73 /*-----*/
74
75 void TaskDigitalRead( void *pvParameters
76     __attribute__((unused)) )
77 // This is a Task.
78 {
79     /*
80     DigitalReadSerial
81     Reads a digital input on pin 2, prints the result to the
82 serial monitor
83     */
84
85     // digital pin 2 has a pushbutton attached to it.
86     uint8_t pushButton = 2;
87
88     // make the pushbutton's pin an input:
89     pinMode(pushButton, INPUT);
90
91     for (;;) // A Task shall never return or exit.
92     {
93         // read the input pin:
94         int buttonState = digitalRead(pushButton);
95
96         /* See if we can obtain or "Take" the Serial Semaphore.
97         If the semaphore is not available, wait 5 ticks of the
98 Scheduler
99         to see if it becomes free*/

```

```

100     if ( xSemaphoreTake( xSerialSemaphore, ( TickType_t ) 5 )
101 == pdTRUE )
102     {
103         /* We were able to obtain or "Take" the semaphore and
104 can now
105     access the shared resource. We want to have the Serial
106 Port for us
107     alone, as it takes some time to print, so we don't want
108 it getting
109     stolen during the middle of a conversion.*/
110     // print out the state of the button:
111     Serial.println(buttonState);
112     xSemaphoreGive( xSerialSemaphore );
113     // Now free or "Give" the Serial Port for others.
114     }
115     vTaskDelay(1);
116     }
117 }
118
119 void          TaskAnalogRead(          void          *pvParameters
120 __attribute__((unused)) )
121 // This is a Task.
122 {
123     for (;;)
124     {
125         // read the input on analog pin 0:
126         int sensorValue = analogRead(A0);
127
128         /* See if we can obtain or "Take" the Serial Semaphore.
129 If the
130     semaphore is not available, wait 5 ticks of the Scheduler
131 to see if
132     it becomes free*/
133         if ( xSemaphoreTake( xSerialSemaphore, ( TickType_t ) 5 )
134 == pdTRUE )
135         {
136             /* We were able to obtain or "Take" the semaphore and
137 can
138     now access the shared resource. We want to have the
139 Serial Port for
140     us alone, as it takes some time to print, so we don't
141 want it
142     getting stolen during the middle of a conversion.*/
143             // print out the value you read:
144             Serial.println(sensorValue);
145             xSemaphoreGive( xSerialSemaphore );
146             // Now free or "Give" the Serial Port for others.
147             }
148             vTaskDelay(1);
149             // one tick delay (15ms) in between ADC reads for
150 stability
151         }
152     }

```

B. Penjelasan Kode

Percobaan43.ino

```
#include <Arduino_FreeRTOS.h> Memasukkan pustaka FreeRTOS untuk menjalankan
sistem operasi waktu nyata (multitasking).
#include <semphr.h> Memasukkan pustaka khusus untuk menggunakan fitur Semaphore
(pengatur hak akses sumber daya bersama).
SemaphoreHandle_t xSerialSemaphore; Mendeklarasikan variabel xSerialSemaphore
sebagai pengendali (handle) Mutex Semaphore untuk port Serial.
void TaskDigitalRead( void *pvParameters ); Mendeklarasikan tugas (task) untuk
membaca input tombol digital.
void TaskAnalogRead( void *pvParameters ); Mendeklarasikan tugas (task) untuk
membaca input sensor analog.
void setup() Fungsi inisialisasi yang dijalankan sekali saat Arduino menyala.
Serial.begin(9600); Memulai komunikasi serial ke komputer pada kecepatan 9600 baud
rate.
while (!Serial) Menunggu hingga port serial berhasil tersambung.
if ( xSerialSemaphore == NULL ) Memeriksa apakah Mutex Semaphore untuk Serial
belum dibuat sebelumnya.
xSerialSemaphore = xSemaphoreCreateMutex(); Membuat Mutex Semaphore untuk
mengatur agar port Serial hanya diakses oleh satu tugas pada satu waktu bergantian.
if ( ( xSerialSemaphore ) != NULL ) Memastikan Mutex Semaphore telah sukses dibuat.
xSemaphoreGive( ( xSerialSemaphore ) ); Memberikan (Give) status awal Semaphore
agar sumber daya Serial terbuka/siap digunakan pertama kali oleh tugas.
xTaskCreate( Memanggil fungsi untuk membuat tugas (task) pertama.
TaskDigitalRead Menugaskan fungsi TaskDigitalRead untuk dieksekusi.
, "DigitalRead" Memberi label nama "DigitalRead" pada tugas ini.
, 128 Mengalokasikan ukuran memori stack sebesar 128 word.
, NULL Tidak mengirim parameter tambahan ke dalam tugas.
, 2 Menetapkan prioritas tugas menjadi 2 (lebih tinggi dari task pembacaan analog).
, NULL ); Tidak menggunakan variabel penunjuk (handle) untuk tugas ini.
```

`xTaskCreate(` Memanggil fungsi untuk membuat tugas (task) kedua.
`TaskAnalogRead` Menugaskan fungsi `TaskAnalogRead` untuk dieksekusi.
`, "AnalogRead"` Memberi label nama "AnalogRead".
`, 128` Mengalokasikan memori stack sebesar 128 word.
`, NULL` Tidak mengirim parameter tambahan.
`, 1` Menetapkan prioritas tugas menjadi 1.
`, NULL`); Tidak menggunakan penunjuk (handle).
`void loop()` Fungsi perulangan utama bawaan Arduino (kosong karena perulangan diatur oleh tugas FreeRTOS).
`void TaskDigitalRead( void *pvParameters __attribute__((unused)) )` Definisi fungsi tugas pembacaan digital (menggunakan `__attribute__((unused))` agar tidak muncul warning karena parameter tidak dipakai).
`uint8_t pushButton = 2;` Menyimpan nomor pin 2 ke dalam variabel `pushButton`.
`pinMode(pushButton, INPUT);` Mengatur pin 2 agar berfungsi sebagai input.
`for (;;)` Perulangan tanpa batas agar tugas ini berjalan terus menerus.
`int buttonState = digitalRead(pushButton);` Membaca status (HIGH/LOW) dari pin 2 dan menyimpannya di variabel `buttonState`.
`if ( xSemaphoreTake( xSerialSemaphore, ( TickType_t ) 5 ) == pdTRUE )` Mencoba mengambil (Take) hak akses Semaphore Serial dengan batas waktu tunggu 5 tick. Jika berhasil (`pdTRUE`), kode di bawahnya dieksekusi.
`Serial.println(buttonState);` Mencetak nilai status tombol ke Serial Monitor secara aman karena sudah memegang hak akses eksklusif.
`xSemaphoreGive( xSerialSemaphore );` Melepaskan (Give) hak akses Semaphore agar port Serial dapat digunakan oleh tugas yang lain.
`vTaskDelay(1);` Menjeda tugas selama 1 tick untuk stabilitas siklus FreeRTOS.
`void TaskAnalogRead( void *pvParameters __attribute__((unused)) )` Definisi fungsi tugas pembacaan analog.
`for (;;)` Perulangan tanpa batas untuk tugas ini.
`int sensorValue = analogRead(A0);` Membaca nilai tegangan dari pin analog A0 dan menyimpannya di variabel `sensorValue`.
`if ( xSemaphoreTake( xSerialSemaphore, ( TickType_t ) 5 ) == pdTRUE )` Mencoba mengambil hak akses Semaphore Serial dengan batas waktu tunggu maksimal 5 tick.

Serial.println(sensorValue); Mencetak nilai pembacaan sensor analog ke Serial Monitor secara eksklusif tanpa tertimpa tugas lain.

xSemaphoreGive(xSerialSemaphore); Melepaskan hak akses Semaphore untuk port Serial.

vTaskDelay(1); Memberikan jeda waktu 1 tick (sekitar 15ms) sebelum membaca konversi analog (ADC) berikutnya.

C. Analisis 4.3

1. *Lakukan perubahan nilai input pada Arduino dengan memutar potensiometer (dan mengamati hasil pembacaannya pada serial monitor), sekaligus melakukan penekanan berulang-ulang pada push button, dan mengamati hasil pembacaan pada serial monitor*

Jawab:

Sistem tetap merespon perubahan potensiometer dan penekanan push button dengan sangat cepat dan realtime seperti pada percobaan 4.2. Bedanya, proses pengiriman data ke serial monitor terasa lebih teratur dan terkelola dengan baik meski ada dua sumber data yang berbeda.

2. *Jelaskan bagaimana tampilan resource serial monitor dalam menangani 2 task sekaligus*

Jawab:

Tampilan pada serial monitor terlihat sangat rapi dan tidak ada teks yang saling tabrakan atau terpotong di tengah jalan. Hal ini karena Semaphore bertindak sebagai semacam "kunci" atau penjaga gawang untuk port Serial. Jika task Analog sedang menulis/print ke serial, task Digital tidak bisa menyela dan harus antri menunggu giliran sampai task Analog selesai dan melepaskan kuncinya, begitu juga sebaliknya.

Kesimpulan

Percobaan 4.1

Penjadwalan multitasking tanpa RTOS yang mengandalkan fungsi delay() biasa sangat tidak efisien untuk sistem yang butuh responsivitas tinggi. Program dieksekusi secara sekuensial, sehingga jika ada satu task yang memakan waktu tunggu lama, task lainnya akan terbengkalai dan menyebabkan sistem menjadi tidak responsif.

Percobaan 4.2

Penggunaan FreeRTOS membuat penjadwalan multitasking jauh lebih optimal dan responsif. Fungsi vTaskDelay memungkinkan mikrokontroler melakukan context switching, di mana waktu jeda sebuah task dimanfaatkan untuk menjalankan task lain, sehingga menghindari terjadinya hambatan pada keseluruhan sistem.

Percobaan 4.3

Fitur Semaphore pada RTOS sangat krusial digunakan ketika ada dua task atau lebih yang ingin mengakses satu sumber daya yang sama, seperti port Serial. Semaphore mencegah terjadinya bentrok atau kerusakan data dengan memastikan sumber daya tersebut hanya bisa diakses secara eksklusif oleh satu task secara bergantian.



LABORATORIUM PEMBELAJARAN ILMU KOMPUTER
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

BAB : RTOS
NAMA : MUHAMMAD NAUFAL MATHARA RAHMAN
NIM : 245150307111006
TANGGAL : 22/04/2026
ASISTEN : - MUHAMMAD WAHYU CAHYO PRASETYO
- DAVIN KENAZ WIDIANANDA TAPPO

TUGAS

-
1. Buat koding (bebas) untuk melakukan penjadwalan multitask, minimal 4 task dengan periode berbeda-beda dan memanfaatkan semaphore untuk mengakses resource yang sama. Lampirkan link video YouTube/drive untuk membuktikan bagaimana jalannya kode program!

Jawab:

<https://youtu.be/Scs1I3ysLXE?si=X2nuBtk02M1PSliP>